COURSE NAME

SOFTWARE ENGINEERING

CSC 3114

(UNDERGRADUATE)

CHAPTER 3

REQUIREMENTS ENGINEERING

ARJUN KUMAR BOSE ARNOB
LECTURER, CS, AIUB
arnob@aiub.edu

Table of Contents

- Introduction to Requirements in Software Engineering
- Types of Software Requirements
- Requirements Engineering Phases
- Challenges in Requirements Engineering
- Impact of Poor Requirements on Software Projects
- Boehm’s Law
- User Stories vs SRS
- The Requirements Baseline
- Definition of Shift Left Testing

Introduction to Requirements in SE

- The software requirements are description of features and functionalities of the target system.
- A condition or capability needed by a user to solve a problem or achieve an objective, or a condition or capability that must be met or possessed by a system or system component.
- Essentially, requirements specify what the software must do, how well it must perform and any constraints it must adhere to.
- For software projects to be successful, the Software Development Life Cycle (SDLC) is essential.

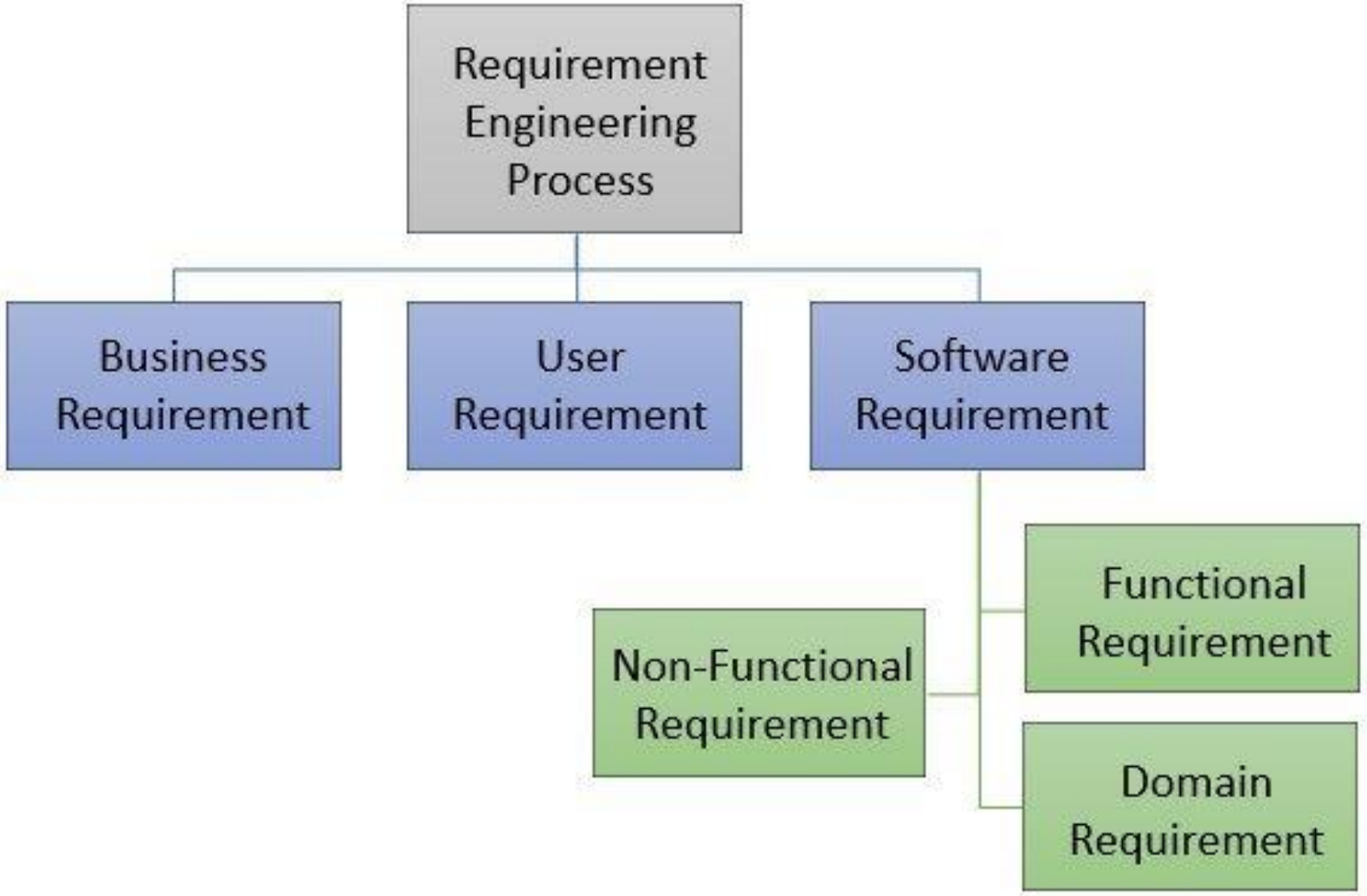


Figure 1: Types of requirements in RE

Role of Requirements in SDLC

- Initial phase of project planning.
- Basis for project estimation and planning.
- Guides design, implementation, testing and maintenance.
- SDLC offers an organized method that guarantees effectiveness, quality and risk reduction all the way through the development process.
- Teams may satisfy client expectations, manage projects more effectively and produce dependable software by conforming to a specified SDLC.

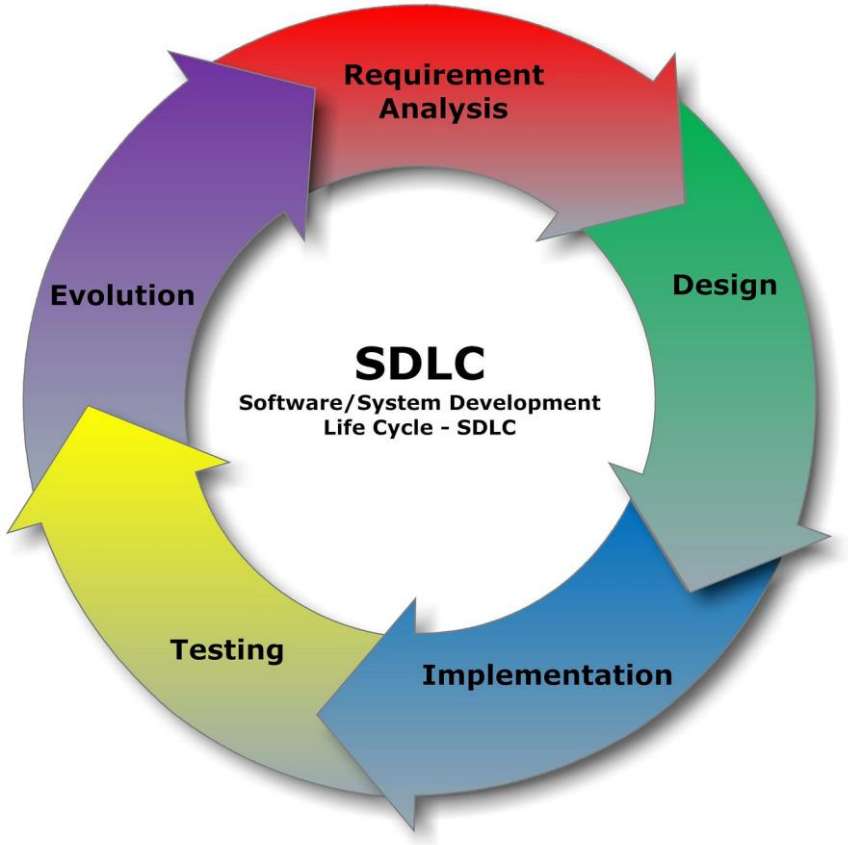


Figure 2: SDLC Phases

User Needs vs. System Requirements

User Needs	System Requirements
1. High-level goals or expectations of the user.	1. Detailed technical specifications the system must fulfill.
2. Described from the user's point of view.	2. Described from the developer's or system designer's perspective.
3. Broad and general.	3. Specific, measurable and testable.
4. To understand what users want or expect.	4. To guide system design, development and testing.

Business vs. Technical Requirements

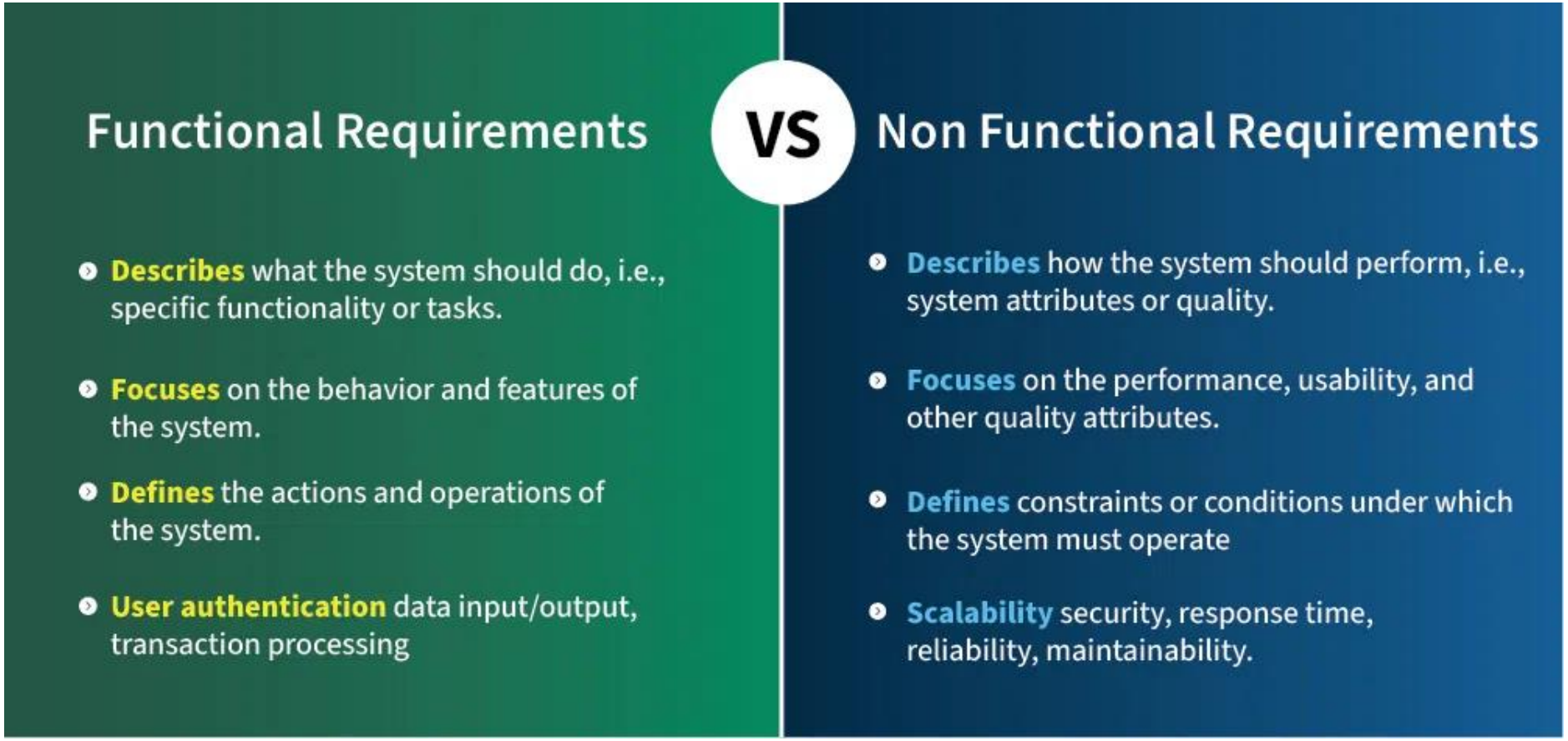
Business Requirements	Technical Requirements
1. High-level goals that the organization wants to achieve.	1. Specific technical solutions to meet business requirements.
2. What the business needs.	2. How the system will fulfill those needs.
3. Defined by business stakeholders (e.g., clients, managers).	3. Defined by technical team (e.g., developers, architects).
4. Increase online sales by 20%, reduce processing time.	4. Use of a secure payment gateway, implementation of necessary APIs.

Functional Requirements and Non-Functional Requirements

- FR defines system behavior and functions.

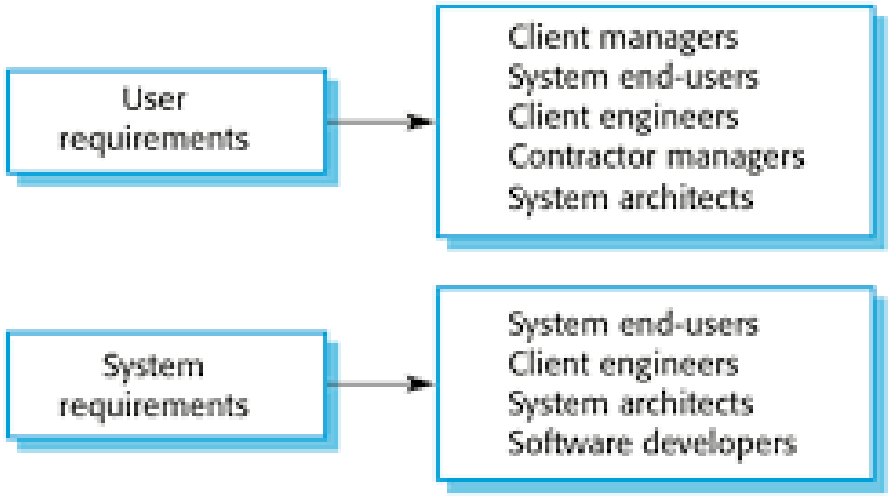
  Example: “System must allow users to log in”.
- NFR defines system attributes like performance, usability.
- Often affect user satisfaction.

Functional Requirements and Non-Functional Requirements [Cont.]



User, System and Business Requirements

- User: What users expect to do.
- System: Detailed technical description.
- Business: Alignment with business goals. From a business standpoint, business requirements specify what the software system must do in order for the company to reach its objectives.



Examples of Critical NFRs

NFR Category	Example
Performance	The system shall handle **1000 transactions per second**.
Reliability	The system shall have **99.99% uptime** over a 12-month period.
Scalability	The system shall support **up to 10,000 concurrent users** without failure.
Security	All user data shall be **encrypted using AES-256** during transmission.
Usability	New users shall be able to learn to use the system within **30 minutes**.

Requirements Engineering Phases

- The phases are:

1. Inception
2. Elicitation
3. Elaboration
4. Negotiation
5. Specification
6. Validation
7. Req. Management



Figure 3: RE Phases

- Cyclical and iterative process

1. Inception

- This phase involves identifying the initial idea and understanding the basic needs and scope of the project.
- It sets the foundation for further detailed analysis and planning.

2. Elicitation

- During elicitation, requirements are gathered from stakeholders using some techniques.
- Understanding stakeholders' real needs.
- Techniques: Interviews, Surveys, Workshops.

3. Elaboration

- In this phase, requirements are analyzed and refined to resolve ambiguities, define detailed specifications and establish a clear understanding of functionalities of the system.

4. Negotiation

- Here, conflicting requirements are resolved through discussions with stakeholders, prioritizing needs and reaching an agreement on the most feasible and valuable features.

5. Specification

- Need to create a documentation based on requirements (like SRS, PRD).
- Clarity, consistency, completeness

6. Validation

- This phase verifies that the documented requirements accurately reflect stakeholder needs and are feasible, ensuring that the system will meet its intended purpose.
- Checking for correctness, feasibility.
- Techniques: Reviews, Prototyping

7. Requirements Management

- Throughout the project, requirements are managed to accommodate changes, track revisions and ensure that the evolving needs are properly incorporated and controlled.

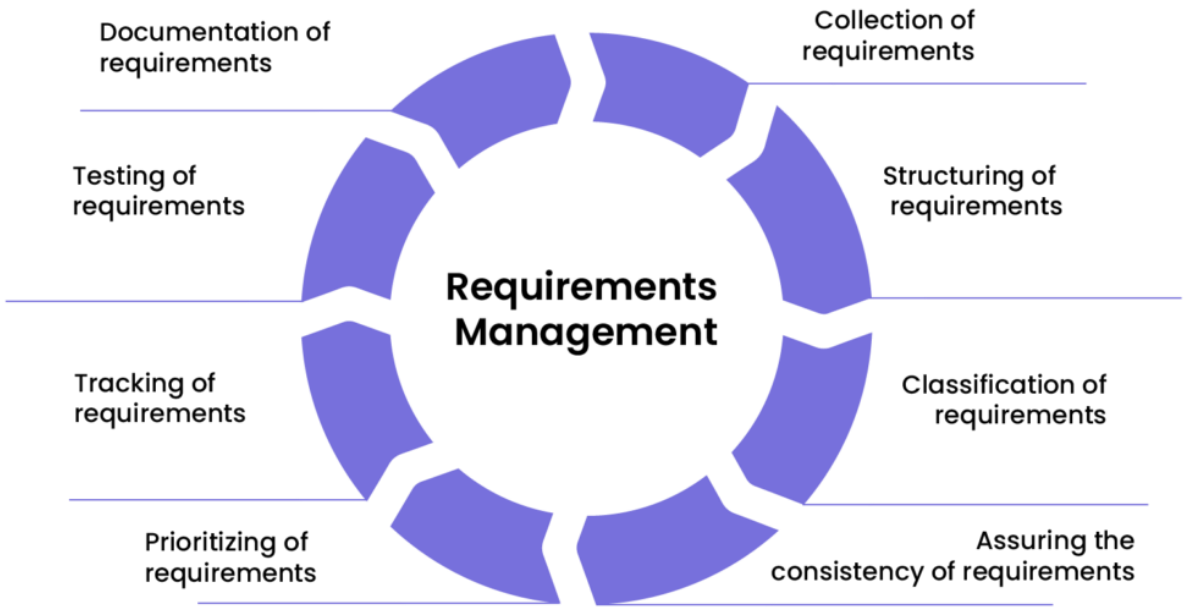


Figure 4: Requirements Management

Challenges in Requirements Engineering

Challenge	Description	Impact
Incomplete Requirements	Some requirements are missing or not well-defined.	Leads to system failures or unmet user expectations.
Changing Requirements	Requirements evolve due to business, market, or user changes.	Causes rework, delays, and increased cost.
Ambiguous Requirements	Requirements are vague or open to multiple interpretations.	Results in incorrect implementation.
Stakeholder Communication Gap	Misunderstandings between developers and non-technical stakeholders.	Misalignment between product and stakeholder needs.
Lack of Domain Knowledge	Development team lacks understanding of the user's environment or business.	Results in irrelevant or impractical system functionality.

Scope Creep

- Uncontrolled changes in requirements.
- Impact: Delays, budget overruns.
- Prevention: Clear documentation, approval processes.

Communication Gaps

- Between stakeholders and developers.
- Causes misunderstandings.
- Solutions: Prototypes, regular meetings.

Impact of Poor Requirements in S/W Projects

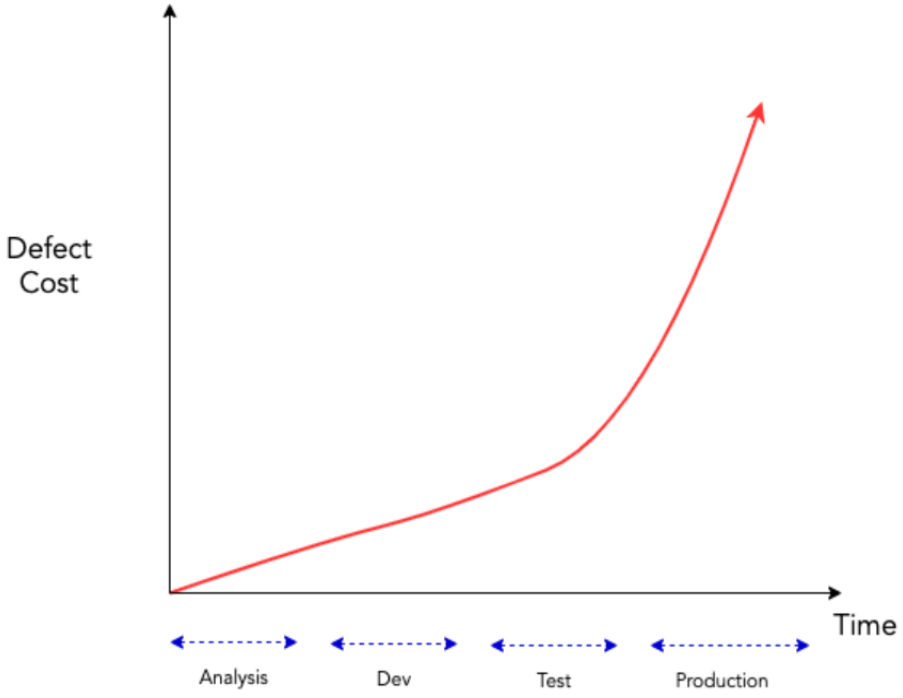
- Inaccurate scope, budget issues.
- Failed projects due to misunderstood requirements.
- Cause misunderstandings and misaligned expectations.
- For example, the FBI Virtual Case File project failed because of vague and incomplete requirements, resulting in a significant budget overrun and eventual cancellation.
- Another case is the Denver International Airport's automated baggage handling system, which was plagued by unclear requirements, leading to extensive delays and cost overruns.

Boehm’s Law

- According to Boehm's Law, commonly referred to as the Law of Costly Errors, the later a software flaw is discovered in the development lifecycle, the more expensive it is to solution.

- To put it simply, it will cost significantly more to fix a bug during testing than it will to cure it during developing and it will cost even more to solve a bug after the software has been deployed.

Boehm's Law (Cont.)



User Stories vs Traditional SRS

- User stories: Simple, customer-focused statements.
- SRS: Software Requirements Specification.

User Stories	Traditional SRS
1. Simple, informal sentence: “As a [user], I want [feature]…”	1. Formal, structured document with detailed sections and specifications.
2. User needs and outcomes.	2. System functionalities and technical details.
3. Iterative, incremental development.	3. Upfront, comprehensive documentation.
4. Lightweight and minimal (Agile: Scrum, XP etc.).	4. Heavyweight, exhaustive documentation (Waterfall, V-model etc.).
5. Highly adaptable to change.	5. Difficult and costly to change once finalized.

The Requirements Baseline

- A requirements baseline is a set of requirements that has been reviewed and agreed upon and serves as the basis for further development.

- A meaningful baselining process gives all the major stakeholders confidence in the following ways:

- Customer management or marketing is confident that the project scope won't explode out of control, because customers manage the scope change decisions.
- User representatives have confidence that the development team will work with them to deliver the right solution, even if they didn't think of every requirement before construction began.
- Development management has confidence because the development team has a business partner who will keep the project focused on achieving its objectives and will work with development to balance schedule, cost, functionality, and quality.
- Business analysts and project managers are confident that they can manage changes to the project in a way that will keep chaos to a minimum.
- Quality assurance and test teams can confidently develop their test scripts and be fully prepared for their project activities.

References

- https://datarob.com/essentials-software-development-life-cycle/
- https://www.geeksforgeeks.org/software-engineering/functional-vs-non-functional-requirements/
- https://dribbble.com/shots/4715049-Requirement-Engineering-Process
- https://xebrio.com/what-is-requirements-management-a-complete-guide/
- https://www.geeksforgeeks.org/software-engineering/software-engineering-challenges-eliciting-requirements/
- https://rnjn.in/glossary/boehms-law/
- https://www.atlassian.com/devops/what-is-devops/agile-vs-devops
- https://www.agilebusiness.org/dsdm-project-framework/moscow-prioririsation.html